B

虚拟机监视器

B.1 引言

多年前, IBM 把昂贵的大型机卖给大型机构, 于是出现了一个问题: 如果机构想在同一台机器上同时运行不同的操作系统, 该怎么办? 有些应用程序是在一种操作系统上开发的, 另一些则是在别的操作系统上开发的, 问题就来了。作为解决办法, IBM 又引入了一个间接层（level of indirection）, 那就是**虚拟机监视器**（virtual machine monitor, **VMM**, 也叫 **hypervisor**）[G74]。

具体来说, 监视器位于一个或多个操作系统和硬件之间, 让每个运行中的操作系统都以为自己掌控着这台机器。而在幕后, 真正掌控硬件的是监视器, 它必须让多个运行中的操作系统复用（multiplex）机器的物理资源。实际上, VMM 就是操作系统的操作系统, 只不过处在低得多的层次上; 操作系统仍然必须认为自己在和物理硬件打交道。因此, **透明**（transparency）是 VMM 的一个主要目标。

这样一来, 我们的处境就有些滑稽了: 到目前为止, 操作系统一直扮演着魔术大师的角色, 让毫不知情的应用程序以为自己拥有私有的 CPU 和很大的虚拟内存（virtual memory）, 同时又在暗中切换应用程序、让它们共享内存。现在, 我们要再来一次, 只不过这次是在操作系统的下面, 而操作系统早已习惯了自己说了算。VMM 怎样才能为运行在它上面的每个操作系统营造出这种假象呢?

关键问题:
如何虚拟化操作系统下面的机器
虚拟机监视器（virtual machine monitor）必须透明地虚拟化（virtualize）操作系统下面的机器; 这需要用到哪些技术?

B.2 动机: 为什么需要 VMM

如今, VMM 出于多种原因再次流行起来。服务器整合（server consolidation）就是其中之一。在很多场合, 人们把服务运行在不同的机器上, 这些机器运行着不同的操作系统（甚至不同版本的操作系统）, 而每台机器的利用率（utilization）都很低。这种情况下, 管理员

可以借助虚拟化把多个操作系统**整合**（consolidate）到更少的硬件平台上，从而降低成本、简化管理。

虚拟化在桌面上也流行起来，因为许多用户希望运行一种操作系统（比如 Linux 或 Mac OS X），同时又能使用另一个平台（比如 Windows）上的原生应用程序。这种**功能**（functionality）上的提升也是一个很好的理由。

另一个原因是测试和调试。开发者在一个主要平台上写代码，但他们常常需要在软件实际部署的许多不同平台上调试和测试。虚拟化让这件事变得很容易，开发者只用一台机器就能运行许多种类、许多版本的操作系统。

这一轮虚拟化复兴在 20 世纪 90 年代中后期真正兴起，带头的是斯坦福大学 Mendel Rosenblum 教授领导的一组研究人员。他们研究的 Disco [B+97] 是一个面向 MIPS 处理器的虚拟机监视器（virtual machine monitor），是让 VMM 复兴的早期尝试，最终也促使这个团队创办了 VMware [V98]，如今它已是虚拟化技术的市场领导者。在本章中，我们将讨论 Disco 背后的主要技术，并借此理解虚拟化是如何工作的。

B.3 虚拟化 CPU

要在虚拟机监视器（virtual machine monitor）上运行一台**虚拟机**（virtual machine，例如一个操作系统及其应用程序），用到的基本技术是**受限直接执行**（limited direct execution），我们在讨论操作系统如何虚拟化 CPU 时见过这种技术。因此，当我们想在 VMM 上“启动”一个新的操作系统时，只需跳转到它第一条指令的地址，让它开始运行就行了。就这么简单（好吧，差不多）。

假设我们运行在单处理器上，想在两台虚拟机之间（也就是在两个操作系统及其各自的应用程序之间）分时复用（multiplex）处理器。和操作系统在运行中的进程（process）之间切换（即**上下文切换**，context switch）非常相似，虚拟机监视器（virtual machine monitor）必须在运行中的虚拟机之间执行 **machine switch**。也就是说，执行这种切换时，VMM 必须保存一个操作系统的全部机器状态（machine state），包括寄存器、PC，以及上下文切换时不必保存的所有特权硬件状态；再恢复即将运行的虚拟机的机器状态，然后跳转到该虚拟机的 PC，从而完成切换。注意，即将运行的虚拟机的 PC 可能位于操作系统内部，即系统当时正在执行系统调用（system call）；也可能只是位于该操作系统上运行的某个进程中，即一个用户模式（user mode）应用程序。

当运行中的应用程序或操作系统试图执行某种**特权操作**（privileged operation）时，问题就稍微棘手一些了。例如，在采用软件管理 TLB（software-managed TLB）的系统上，操作系统会用专门的特权指令（privileged instruction）把一条转换装入 TLB，然后重新执行那条发生了 TLB 未命中（TLB miss）的指令。在虚拟化环境（virtualized environment）中，不能允许操作系统执行特权指令，否则掌控机器的就是它，而不是它下面的 VMM 了。因此，VMM 必须设法拦截执行特权操作的尝试，从而保住对机器的控制。

VMM 必须介入某些操作，一个简单的例子是：某个操作系统上运行的进程试图发起系统调用。例如，进程可能要对某个文件调用 `open()`，或者调用 `read()` 从中读数据，或者调用 `fork()` 创建新进程。在没有虚拟化的系统中，系统调用是通过一条专门的指令完成的：在 MIPS 上，这是一条**陷阱**（trap）指令；在 x86 上，则是参数为 `0x80` 的 `int` 指令，

 © 2008–23, ARPACI-DUSSEAU

int 就是中断（interrupt）。下面是 FreeBSD 上的 open 库调用 [B00]。回想一下，你的 C 代码先调用 C 库里的函数，C 库再执行相应的汇编指令序列，真正发出陷阱（trap）指令，完成系统调用：

```
open:
    push    dword mode
    push    dword flags
    push    dword path
    mov     eax, 5
    push    eax
    int     80h
```

在基于 UNIX 的系统上，open() 只接受三个参数：int open(char *path, int flags, mode_t mode)。从上面的代码可以看出 open() 库调用是怎么实现的：先把参数（mode、flags、path）压入栈（stack）中，再把 5 压入栈中，然后调用 int 80h，把控制权交给内核。如果你好奇的话，这个 5 是 FreeBSD 中用户模式应用程序和内核为 open() 系统调用事先约定好的编号；不同的系统调用会在调用陷阱（trap）指令 int 之前，把不同的编号压入栈中（放在同一位置），以此发起系统调用[1]。

我们之前讨论过，执行陷阱（trap）指令时通常会发生不少有意思的事。在这个例子里，最重要的一点是：它首先把控制权转交（即修改 PC）给操作系统中一个确定的**陷阱处理程序**（trap handler）。操作系统刚启动时，会把这个例程的地址告诉硬件，这本身也是一项特权操作；这样，此后再发生陷阱（trap）时，硬件就知道从哪里开始运行处理陷阱的代码。在发生陷阱（trap）的同时，硬件还会做另一件至关重要的事：把处理器从**用户模式**（user mode）切换到**内核模式**（kernel mode）。在用户模式下，操作是受限的，试图执行特权操作会引发陷阱（trap），出错的进程多半会被终止；而在内核模式下，机器的全部能力都可以使用，所有特权操作都能执行。因此，在传统环境下——再说一遍，没有虚拟化——控制流程如图 B.1 所示。

在虚拟化平台（virtualized platform）上，事情就有意思一些了。运行在某个操作系统上的应用程序想发起系统调用时，做的事情完全一样：把参数小心地放到栈上或寄存器里，然后执行陷阱（trap）指令。然而，现在掌控机器的是 VMM，安装陷阱处理程序（trap handler）的也是 VMM，所以首先在内核模式下执行的是 VMM 的处理程序。

那么，VMM 该怎么处理这个系统调用呢？VMM 其实并不知道**怎样**处理这个调用；毕竟，它不了解每个运行中的操作系统的细节，因此也就不知道每个调用该做什么。不过，VMM 确实知道操作系统的陷阱处理程序（trap handler）在**哪里**。它之所以知道，是因为操作系统启动时曾试图安装自己的陷阱处理程序（trap handler）；操作系统这么做时，是在尝试执行特权操作，因此陷入了 VMM；那时 VMM 就记下了必要的信息，也就是这个操作系统的陷阱处理程序（trap handler）在内存中的位置。现在，当 VMM 收到某个操作系统上运行的用户进程（user process）发来的陷阱（trap）时，它就知道该怎么做了：跳转到该操作系统的陷阱处理程序（trap handler），让操作系统按照应有的方式处理系统调用。操作

[1] 让事情更乱的是，Intel 的人把几乎任何正常人都会称为陷阱（trap）指令的东西叫作“中断”（interrupt）。正如 Patterson 评价 Intel 指令集时所说：“这是一套只有亲妈才会爱的 ISA。”不过说实话，我们还挺喜欢它的，而我们又不是它的妈。

进程	硬件	操作系统
1. 执行指令（add、load 等）		
2. 系统调用：陷入操作系统		
	3. 切换到内核模式；跳转到陷阱处理程序	
		4. 在内核模式下：处理系统调用；从陷阱返回
	5. 切换到用户模式；返回用户代码	
6. 继续执行（陷阱之后的 PC 处）		

图 B.1：**执行系统调用**

系统处理完后，会执行某种特权指令从陷阱返回（MIPS 上是 **`rett`**，x86 上是 `iret`），这又会弹回到 VMM；VMM 意识到操作系统正试图从陷阱返回（return-from-trap），于是执行一次真正的从陷阱返回，把控制权交还给用户，并让机器回到用户模式。图 B.2 和图 B.3 分别描绘了没有虚拟化和有虚拟化两种情况下的完整过程（为了节省篇幅，我们省略了上面那些具体的硬件操作）。

从图中可以看出，有虚拟化时要做的事情多得多。毫无疑问，由于多了这些来回跳转，虚拟化确实可能拖慢系统调用，从而影响性能。

你可能还注意到，还剩一个问题：操作系统应该运行在什么模式下？它不能运行在内核模式下，否则它就能不受限制地访问硬件了。因此，它必须运行在某种比以前特权低的模式下，既能访问自己的数据结构（data structure），又要防止用户进程（user process）访问这些数据结构。

在 Disco 的工作中，Rosenblum 和同事们利用 MIPS 硬件提供的一种叫管理模式（supervisor mode）的特殊模式，相当巧妙地解决了这个问题。在这种模式下运行时，仍然不能执行特权指令，但可以访问的内存比用户模式多一些；操作系统可以把这些额外的内存

进程	操作系统
1. 系统调用：陷入操作系统	
	2. 操作系统的陷阱处理程序：解码陷阱，执行相应的系统调用例程；完成后：从陷阱返回
3. 继续执行（陷阱之后的 PC 处）	

图 B.2：**没有虚拟化时的系统调用流程**

进程	操作系统	VMM
1. 系统调用： 陷入操作系统		
		2. 进程陷入： 调用操作系统的陷阱处理程序 （以较低特权运行）
	3. 操作系统的陷阱处理程序： 解码陷阱， 执行系统调用； 完成后：发出 从陷阱返回指令	
		4. 操作系统试图从陷阱返回： 执行真正的从陷阱返回
5. 继续执行 （陷阱之后的 PC 处）		

图 B.3：**有虚拟化时的系统调用流程**

用于自己的数据结构，问题就解决了。在没有这种模式的硬件上，就只能让操作系统运行在用户模式下，并借助页表（page table）和 TLB 做内存保护（memory protection），恰当地保护操作系统的数据结构。换句话说，切换到操作系统时，监视器得通过页表保护让操作系统能够访问其数据结构所在的内存；切换回运行中的应用程序时，又得撤销读写内核的权限。

B.4 虚拟化内存

现在你应该基本了解了处理器是怎样虚拟化的：VMM 像操作系统一样调度不同的虚拟机运行，而在特权级别发生变化时会出现一些有意思的交互。但我们还漏掉了一大块：VMM 是怎样虚拟化内存的?

每个操作系统通常都把物理内存（physical memory）看作一个由页（page）组成的线性数组，并把每一页分配给自己或用户进程（user process）。当然，操作系统本身已经在为它运行的进程虚拟化内存了，让每个进程都以为自己拥有私有的地址空间（address space）。现在我们必须再加一层虚拟化，让多个操作系统共享机器真正的物理内存，而且必须做得透明。

有了这额外的一层虚拟化，“物理”内存本身就成了一种虚拟化，建立在 VMM 所说的**机器内存**（machine memory）之上；机器内存才是系统真正的物理内存。因此，现在多了一个间接层（level of indirection）：每个操作系统通过各进程的页表把虚拟地址（virtual address）映射到物理地址（physical address）；VMM 再通过每个操作系统各自的页表，把得到的物理地址映射到底层的机器地址（machine address）。图 B.4 描绘了这个额外的间接层（level of indirection）。

图中只有一个虚拟地址空间（virtual address space），它有四个页，其中三个是有效的（0、2 和 3）。操作系统用它的页表把这几页分别映射到底层的物理页帧（physical frame）10、3 和 8。在操作系统下面，VMM 又做了一层间接映射，把 PFN 3、8 和 10 分别映射到

 © 2008–23, ARPACI-DUSSEAU

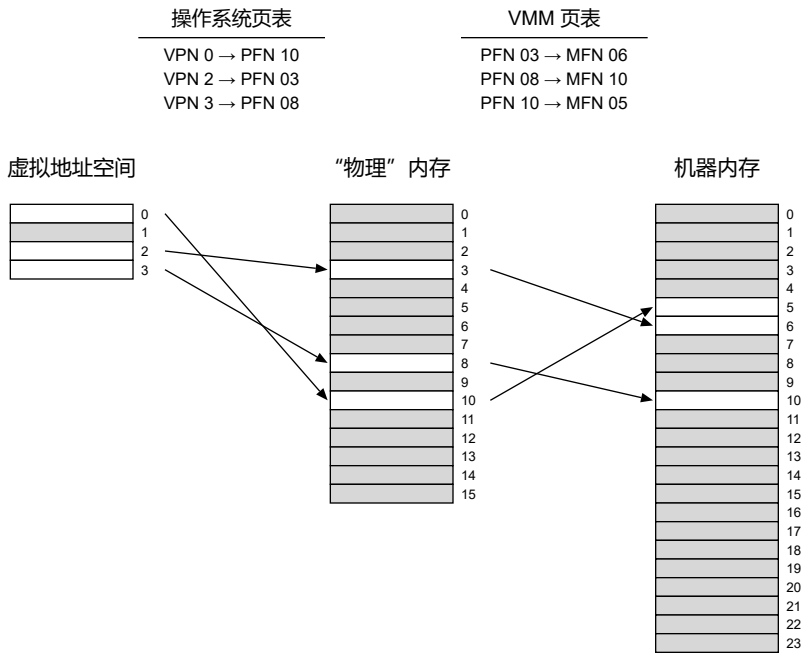


图 B.4：**VMM 的内存虚拟化**

机器页帧（machine frame）6、10 和 5。当然，这张图把事情简化了不少；在真实的系统上，会有 V 个操作系统在运行（V 很可能大于 1），因此有 V 张 VMM 页表；而且，在每个运行中的操作系统 OSi 之上，还运行着若干个进程 P_i（P_i 很可能有几十甚至几百个），因此 OSi 中有 P_i 张（每进程一张）页表。

为了更好地理解这是怎么工作的，我们来回顾一下现代分页（paging）系统中**地址转换**（address translation）是怎样进行的。具体来说，我们来讨论在采用软件管理 TLB（software-managed TLB）的系统上，地址转换时会发生什么。假设某个用户进程产生了一个地址（用于取指令，或者显式的加载或存储）；按照定义，进程产生的是**虚拟地址**（virtual address），因为它的地址空间已经被操作系统虚拟化了。你现在已经知道，操作系统的职责就是在硬件的帮助下，把它转换成**物理地址**（physical address），从而能从物理内存中取出所需的内容。

假设虚拟地址空间是 32 位的，页大小（page size）为 4 KB。于是，32 位的地址被切成两部分：20 位的虚拟页号（VPN）和 12 位的偏移量（offset）。操作系统的职责是在硬件 TLB 的帮助下，把 VPN 转换成有效的物理帧号（PFN），从而得到一个完整的物理地址，再发给物理内存去取出正确的数据。通常情况下，我们期望 TLB 在硬件中完成转换，让转换很快。至少在采用软件管理 TLB（software-managed TLB）的系统上，发生 TLB 未命中时，操作系统必须介入，处理这次未命中，如图 B.5 所示。

可以看到，TLB 未命中会引发陷阱（trap），陷入操作系统；操作系统在页表中查找 VPN，把转换装入 TLB，以此处理这次故障。

进程	操作系统
1. 从内存加载: TLB 未命中：陷入	
	2. 操作系统的 TLB 未命中处理程序: 从虚拟地址中取出 VPN; 查页表; 如果存在且有效: 取得 PFN，更新 TLB; 从陷阱返回
3. 继续执行 (引发陷阱的那条指令的 PC 处); 重新执行这条指令; 这次 TLB 命中	

图 B.5：**没有虚拟化时的 TLB 未命中流程**

然而，如果操作系统下面还有一个虚拟机监视器（virtual machine monitor），事情又会变得有意思一些。我们再来看一遍 TLB 未命中的流程（总结见图 B.6）。进程发出一次虚拟内存引用而在 TLB 中未命中时，运行的不是操作系统的 TLB 未命中处理程序（TLB miss handler），而是 VMM 的 TLB 未命中处理程序，因为 VMM 才是这台机器真正拥有特权的主人。不过，通常情况下，VMM 的 TLB 处理程序不知道该怎么处理这次 TLB 未命中，所以它立即跳转到操作系统的 TLB 未命中处理程序（TLB miss handler）；VMM 知道这个处理程序的位置，因为操作系统在“启动”时曾试图安装自己的陷阱处理程序（trap handler）。接着，操作系统的 TLB 未命中处理程序（TLB miss handler）开始运行，在页表中查找相应的 VPN，并试图把 VPN 到 PFN 的映射装入 TLB。然而，这是一项特权操作，因此又会陷入 VMM（当然，只要非特权代码试图做特权的事，VMM 都会收到通知）。这时，VMM 就耍了个花招：它不装入操作系统给出的 VPN 到 PFN 的映射，而是装入它自己想要的 VPN 到 MFN 的映射。这之后，系统最终回到用户级代码，重新执行那条指令，这次 TLB 命中（TLB hit），从数据所在的机器页帧（machine frame）中取出数据。

这一系列动作也暗示了 VMM 必须怎样为每个运行中的操作系统管理物理内存的虚拟化：就像操作系统为每个进程维护一张页表一样，VMM 必须为它运行的每台虚拟机记录物理地址到机器地址（machine address）的映射。VMM 的 TLB 未命中处理程序（TLB miss handler）需要查询这些每台虚拟机一张的页表，才能确定某个“物理”页映射到哪个机器页（machine page），甚至确定它此刻是否在机器内存（machine memory）中，因为 VMM 可能已经把它换出（swap out）到了磁盘上。

最后，你可能已经从这一系列操作中注意到，在虚拟化系统（virtualized system）上，TLB 未命中的代价比非虚拟化系统高出不少。为了降低这个代价，Disco 的设计者在 VMM 层面加了一个“软件 TLB”（software TLB）。这个数据结构背后的思想很简单：操作系统每次试图装入一条虚拟地址到物理地址的映射，VMM 都把它记下来；然后，发生 TLB 未命中时，VMM 先查一查它的软件 TLB，看看以前是否见过这个虚拟地址到物理地址的映射，以及 VMM 想要的虚拟地址到机器地址（machine address）的映射应该是什么。如果 VMM

 © 2008–23, ARPACI-DUSSEAU

进程	操作系统	虚拟机监视器
1. 从内存加载 TLB 未命中：陷入		
		2. VMM 的 TLB 未命中处理程序： 调用操作系统的 TLB 处理程序 （降低特权）
	3. 操作系统的 TLB 未命中处理程序： 从虚拟地址中取出 VPN； 查页表； 如果存在且有效， 取得 PFN，更新 TLB	
		4. 陷阱处理程序： 非特权代码试图 更新 TLB； 操作系统试图装入 VPN 到 PFN 的映射； 改为用 VPN 到 MFN 的映射 更新 TLB（特权操作）； 跳回操作系统 （降低特权）
	5. 从陷阱返回	
		6. 陷阱处理程序： 非特权代码试图 从陷阱返回； 从陷阱返回
7. 继续执行 （那条指令的 PC 处）； 重新执行这条指令； 这次 TLB 命中		

图 B.6：**有虚拟化时的 TLB 未命中流程**

补充：hypervisor 与硬件管理的 TLB

前面的讨论都围绕软件管理的 TLB（software-managed TLB），以及发生未命中时要做的工作。但你可能会好奇：在采用硬件管理 TLB（hardware-managed TLB）的系统上，虚拟机监视器（virtual machine monitor）怎样介入呢? 在这类系统上，每次 TLB 未命中时由硬件遍历页表并按需更新 TLB，因此 VMM 没有机会在每次 TLB 未命中时运行，把自己的转换偷偷塞进系统。相反，VMM 必须密切监视操作系统对每张页表所做的修改——在硬件管理的系统上，页表由某种页表基址寄存器（page-table base register）指向——并维护一张**影子页表**（shadow page table），把每个进程的虚拟地址映射到 VMM 想要的机器页（machine page）上 [AA06]。每当操作系统试图装入某个进程的操作系统级页表时，VMM 就装入该进程的影子页表（shadow page table）；于是硬件照常运转，用影子页表（shadow page table）把虚拟地址转换成机器地址（machine address），而操作系统甚至毫无察觉。

补充：半虚拟化

在很多情况下，最好假设没法为了和虚拟机监视器（virtual machine monitor）更好地配合而去修改操作系统（例如，因为你是在一家不友好的竞争对手的操作系统下面运行你的 VMM）。然而，情况并非总是如此。正如我们在页的按需置零（demand zeroing）那个例子中看到的，如果操作系统可以修改，它在 VMM 上就可能运行得更高效。让修改过的操作系统运行在 VMM 上，一般称为**半虚拟化**（para-virtualization）[WSG02]，因为 VMM 提供的虚拟化并不完整，而是部分的，需要修改操作系统才能有效运行。研究表明，一个设计得当的半虚拟化系统（para-virtualized system），只要对操作系统做恰到好处的修改，效率就能做到几乎和没有 VMM 的系统一样高 [BD+03]。

在软件 TLB 中找到了这条转换，就直接把虚拟地址到机器地址（machine address）的映射装入硬件 TLB，从而省去上面控制流程中的那一大堆来回 [B+97]。

B.5 信息鸿沟

操作系统不太清楚应用程序真正想要什么，因此常常只能制定一些但愿对所有程序都适用的通用策略；同样，VMM 往往也不太清楚操作系统在做什么、想要什么。这种了解上的欠缺，有时被称为 VMM 和操作系统之间的**信息鸿沟**（information gap），可能导致各种低效 [B+97]。例如，操作系统在没有别的事可做时，有时会进入**空闲循环**（idle loop），只是在那里空转，等待下一次中断发生：

```
while (1)
  ; // 空闲循环
```

如果整台机器归操作系统掌管，它也因此知道没有别的事情需要运行，那这样空转是说得通的。然而，如果 VMM 下面运行着两个不同的操作系统，一个处在空闲循环（idle loop）中，另一个正在运行用户进程（user process）做有用的事，那么 VMM 最好能知道前一个操作系统处于空闲状态，这样就能把更多 CPU 时间分给正在做有用工作的那个操作系统。

另一个例子出现在页的按需置零（demand zeroing）上。大多数操作系统在把一个物理页帧（physical frame）映射到某个进程的地址空间之前，都会先把它清零。这样做的原因很简单：安全。如果操作系统把一个进程用过的页不清零就交给另一个进程，就可能发生跨进程的信息泄漏，从而可能泄露敏感信息。遗憾的是，出于同样的原因，VMM 也必须把分给各个操作系统的页清零，所以很多时候一个页会被清零两次：一次是 VMM 把它分配给操作系统时，另一次是操作系统把它分配给进程时。Disco 的作者对这个问题没有什么好办法：他们只是修改了操作系统（IRIX），让它不再清零那些它知道已被底层 VMM 清零过的页 [B+97]。

类似这里所描述的问题还有很多。一种解决办法是让 VMM 利用推断（一种**隐式信息**，implicit information）来克服问题。例如，VMM 发现操作系统切换到了低功耗模式，就能判断它进入了空闲循环（idle loop）。另一种办法见于**半虚拟化**（para-virtualized）系统，它要求修改操作系统。这种更显式的方法虽然更难部署，却可能相当有效。

提示：利用隐式信息

在分层系统中，如果很难修改系统之间的接口，却又需要了解系统另一层的更多信息，隐式信息就会是一个强大的工具。例如，一个基于块（block）的磁盘设备可能想更多地了解它上面的文件系统（file system）是怎样使用它的；同样，一个应用程序可能想知道当前哪些页在文件系统的页缓存（page cache）中，而操作系统没有提供任何 API 来获取这些信息。在这两种情况下，研究人员都已经开发出了强大的推断技术，能够隐式地收集所需的信息，*而不需要*层与层之间有显式的接口 [AD+01, S+03]。这类技术在虚拟机监视器（virtual machine monitor）中相当有用，因为 VMM 想更多地了解运行在它上面的操作系统，又不想在两层之间加一个显式的 API。

B.6 小结

虚拟化正在复兴。出于多种原因，用户和管理员都希望在同一台机器上同时运行多个操作系统。关键在于，VMM 一般能**透明地**（transparently）提供这种服务；上面的操作系统几乎察觉不到自己其实并没有在控制这台机器的硬件。VMM 做到这一点的关键方法，是扩展受限直接执行（limited direct execution）的概念：把硬件设置好，让 VMM 能介入陷阱（trap）这类关键事件，VMM 就能完全控制机器资源的分配方式，同时又保住操作系统所需要的假象。

你可能已经注意到，操作系统为进程做的事和 VMM 为操作系统做的事有一些相似之处。毕竟它们都在虚拟化硬件，所以会做一些相同的事情。然而，两者有一个关键区别：操作系统的虚拟化提供了许多新的抽象（abstraction）和好用的接口；而 VMM 层面的虚拟化，提供的抽象和硬件一模一样（因而并不怎么好用）。操作系统和 VMM 虽然都在虚拟化硬件，但它们提供的接口完全不同；和操作系统不同，VMM 的本意并不是让硬件更容易使用。

如果你想更多地了解虚拟化，还有许多其他话题可以研究。例如，我们根本没有讨论 I/O 的情况，而在虚拟化平台（virtualized platform）上，I/O 有它自己新的、有意思的问题。我们也没有讨论，在有时称为“宿主型”（hosted）的配置下，虚拟化和你的操作系统“并排”运行时是怎样工作的。如果你感兴趣，可以进一步阅读这两个话题 [SVL01]。我们还没有讨论，运行在一个 VMM 上的一组操作系统用了太多内存时会发生什么。

最后，硬件支持已经改变了平台支持虚拟化的方式。Intel 和 AMD 等公司如今直接支持多出来的这一层虚拟化，使得本章中的许多软件技术都不再必要。也许在一个尚待写出的章节中，我们会更详细地讨论这些机制。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[AA06] "A Comparison of Software and Hardware Techniques for x86 Virtualization" by Keith Adams and Ole Agesen. ASPLOS '06, San Jose, California. *A terrific paper from two VMware engineers about the surprisingly small benefits of having hardware support for virtualization. Also an excellent general discussion about virtualization in VMware, including the crazy binary-translation tricks they have to play in order to virtualize the difficult-to-virtualize x86 platform.*

[AD+01] "Information and Control in Gray-box Systems" by Andrea C. Arpaci-Dusseau and Remzi H. Arpaci-Dusseau. SOSP '01, Banff, Canada. *Our own work on how to infer information and even exert control over the OS from application level, without change to the OS. The best example therein: determining which file blocks are cached using a probabilistic probe-based technique; doing so allows applications to better utilize the cache, by first scheduling work that will result in hits.*

[B00] "FreeBSD Developers' Handbook: Chapter 11 x86 Assembly Language Programming" http://www.freebsd.org/doc/en/books/developers-handbook/. *A nice tutorial on system calls and such in the BSD developers handbook.*

[BD+03] "Xen and the Art of Virtualization" by Paul Barham, Boris Dragovic, Keir Fraser, Steven Hand, Tim Harris, Alex Ho, Rolf Neugebauer, Ian Pratt, Andrew Warfield. SOSP '03, Bolton Landing, New York. *The paper that shows that with para-virtualized systems, the overheads of virtualized systems can be made to be incredibly low. So successful was this paper on the Xen virtual machine monitor that it launched a company.*

[B+97] "Disco: Running Commodity Operating Systems on Scalable Multiprocessors" by Edouard Bugnion, Scott Devine, Kinshuk Govil, Mendel Rosenblum. SOSP '97. *The paper that reintroduced the systems community to virtual machine research; well, perhaps this is unfair as Bressoud and Schneider [BS95] also did, but here we began to understand why virtualization was going to come back. What made it even clearer, however, is when this group of excellent researchers started VMware and made some billions of dollars.*

[B+17] "Hardware and Software Support for Virtualization" by Edouard Bugnion, Jason Nieh, Dan Tsafrir. Morgan and Claypool, 2017. *Undoubtedly the best place to get the latest on how virtualization works in modern systems. Unfortunately, you'll have to read a short book to figure it out!*

[BS95] "Hypervisor-based Fault-tolerance" by Thomas C. Bressoud, Fred B. Schneider. SOSP '95. *One the earliest papers to bring back the* **hypervisor***, which is just another term for a virtual machine monitor. In this work, however, such hypervisors are used to improve system tolerance of hardware faults, which is perhaps less useful than some of the more practical scenarios discussed in this chapter; however, still quite an intriguing paper in its own right.*

[G74] "Survey of Virtual Machine Research" by R.P. Goldberg. IEEE Computer, Volume 7, Number 6. *A terrific survey of a lot of old virtual machine research.*

[SVL01] "Virtualizing I/O Devices on VMware Workstation's Hosted Virtual Machine Monitor" by Jeremy Sugerman, Ganesh Venkitachalam and Beng-Hong Lim. USENIX '01, Boston, Massachusetts. *Provides a good overview of how I/O works in VMware using a hosted architecture which exploits many native OS features to avoid reimplementing them within the VMM.*

[V98] by VMware corporation. Available: http://www.vmware.com/. *This may be the most useless reference in this book, as you can clearly look this up yourself. Anyhow, the company was founded in 1998 and is a leader in the field of virtualization.*

[S+03] "Semantically-Smart Disk Systems" by Muthian Sivathanu, Vijayan Prabhakaran, Florentina I. Popovici, Timothy E. Denehy, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '03, San Francisco, California, March 2003. *Our work again, this time showing how a dumb block-based device can infer much about what the file system above it is doing, such as deleting a file. The technology used therein enables interesting new functionality within a block device, such as secure delete, or more reliable storage.*

[WSG02] "Scale and Performance in the Denali Isolation Kernel" by Andrew Whitaker, Marianne Shaw, and Steven D. Gribble. OSDI '02, Boston, Massachusetts. *The paper that introduces the term para-virtualization. Although one can argue that Bugnion et al. [B+97] introduce the idea of para-virtualization in the Disco paper, Whitaker et al. take it further and show how the idea can be more general than what was thought before.*